

Table Of Contents

	1
A. Lab 1: Data Type Size Exploration	
	2
B. Lab 2: Memory Segment Address Mapping	
	5
3) Identify which segment has the highest addresses	6
4) Identify which segment has the lowest addresses	6
5) Calculate the address difference between Stack and Heap	6
Knowledge Check Questions	
	7
1) What is a data type in the context of programming and operating systems?	7
2) Name the five main memory segments in a process memory layout.	7
Answer:	7
3) What is the difference between the Data segment and the BSS segment?	7
4) Why does the OS place the stack at high virtual addresses and the text segment at low addresses?	7
5) What happens to stack memory when a function returns? Who reclaims it?	7
6) How does the OS know which memory belongs to which process?	8
Github Link : https://github.com/prathanaamgain/PrathanaOS_batch38.git	8

Table of Figure

Figure 1: C program that prints the size of processes

Figure 2: Compiled and run

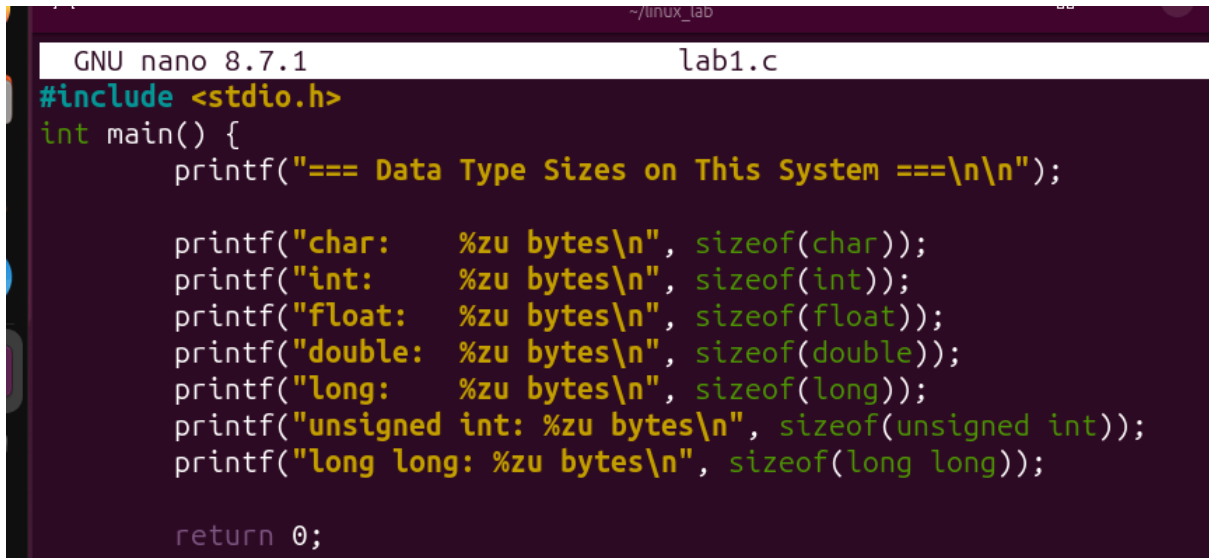
Figure 3: Comparing with Table 1

Figure 4: Code snippet

Figure 5: Compilation and running the code

A. Lab 1: Data Type Size Exploration

1) Create a C program that prints the size of: char, int, float, double, long, unsigned int, long long



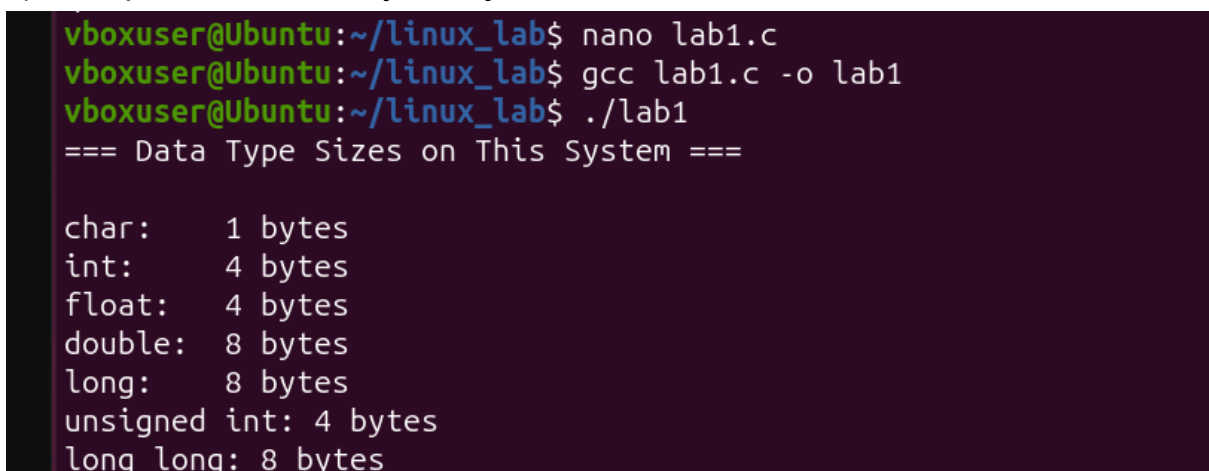
```
GNU nano 8.7.1 lab1.c
#include <stdio.h>
int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char:    %zu bytes\n", sizeof(char));
    printf("int:     %zu bytes\n", sizeof(int));
    printf("float:   %zu bytes\n", sizeof(float));
    printf("double:  %zu bytes\n", sizeof(double));
    printf("long:    %zu bytes\n", sizeof(long));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long long: %zu bytes\n", sizeof(long long));

    return 0;
}
```

Figure1: C program that prints the size of processes

2) Compile and run on your system



```
vboxuser@Ubuntu:~/linux_lab$ nano lab1.c
vboxuser@Ubuntu:~/linux_lab$ gcc lab1.c -o lab1
vboxuser@Ubuntu:~/linux_lab$ ./lab1
=== Data Type Sizes on This System ===

char:    1 bytes
int:     4 bytes
float:   4 bytes
double:  8 bytes
long:    8 bytes
unsigned int: 4 bytes
long long: 8 bytes
```

Figure 2: Compiled and run

3) Compare output with Table 1

Data Type	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 bytes	2 bytes	2 bytes	Memory-efficient counters
int	2 bytes	4 bytes	4 bytes	Affects pointer arithmetic
long	4 bytes	4 bytes	8 bytes	Critical for 64-bit addressing
long long	8 bytes	8 bytes	8 bytes	Large file offsets
float	4 bytes	4 bytes	4 bytes	IEEE 754 standard
double	8 bytes	8 bytes	8 bytes	Precision calculations
pointer	2 bytes	4 bytes	8 bytes	Address space limit

```
vboxuser@Ubuntu:~/linux_lab$ nano lab1.c
vboxuser@Ubuntu:~/linux_lab$ gcc lab1.c -o lab1
vboxuser@Ubuntu:~/linux_lab$ ./lab1
=== Data Type Sizes on This System ===

char:  1 bytes
int:    4 bytes
float:  4 bytes
double: 8 bytes
long:   8 bytes
unsigned int: 4 bytes
long long: 8 bytes
```

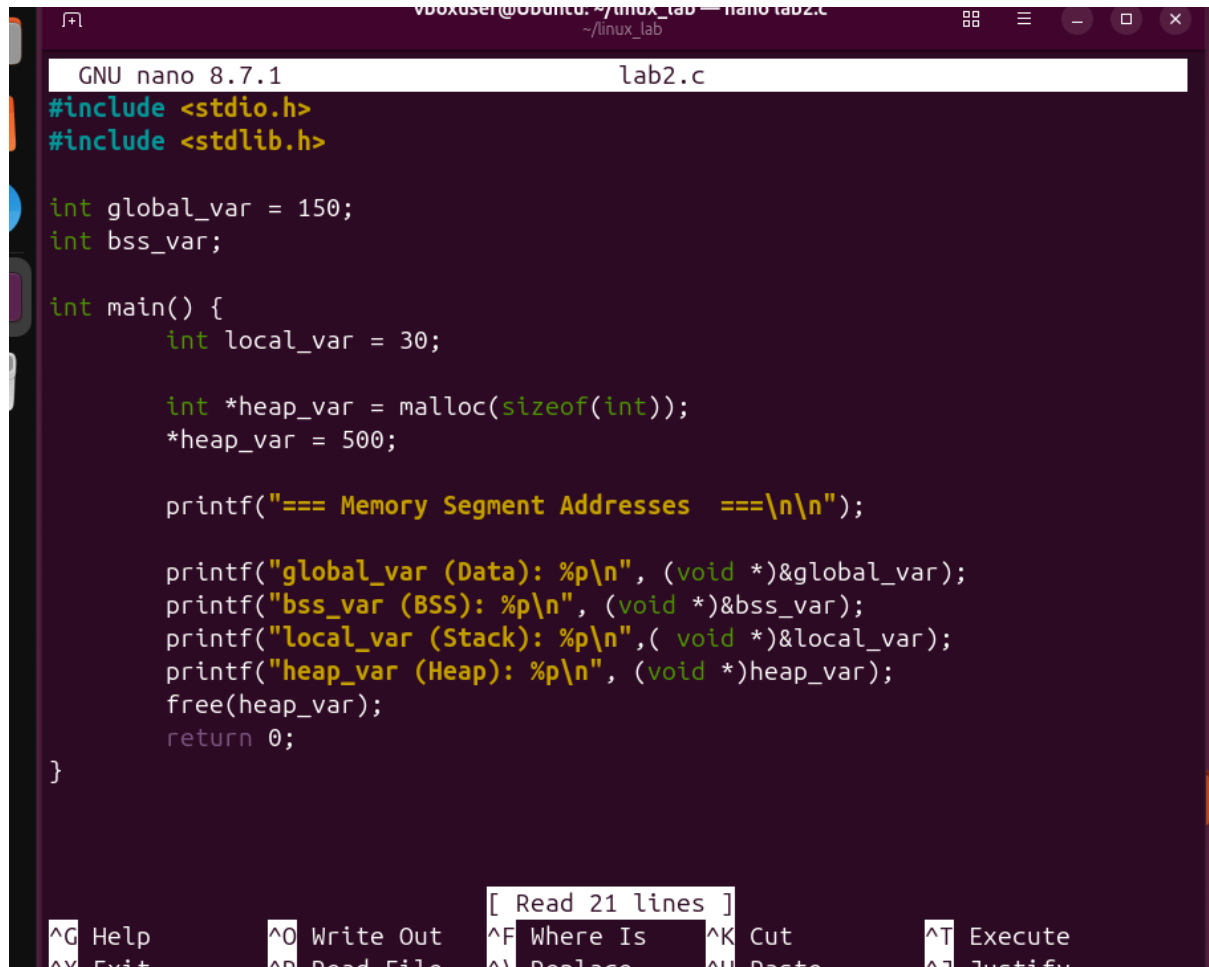
Figure 3: Comparing with Table 1

Summary: The results from the code mostly match Table 1. However, Table 1 shows 8 bytes for a pointer on a 64-bit system, while the program shows 4 bytes because it uses `sizeof(int)` instead of `sizeof(int*)`. `sizeof(int)` measures the size of an integer, whereas `sizeof(int*)` measures the size of a pointer.

4) Why might `sizeof(long)` differ between 32-bit and 64-bit systems?

The size of `sizeof(long)` can vary between 32-bit and 64-bit systems because of differences in the system architecture, including CPU registers and pointer sizes. On a 32-bit system, `long` typically uses 4 bytes, while on a 64-bit Linux/Unix system, it is generally 8 bytes to support the larger address space.

B. Lab 2: Memory Segment Address Mapping



```
GNU nano 8.7.1 lab2.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 150;
int bss_var;

int main() {
    int local_var = 30;

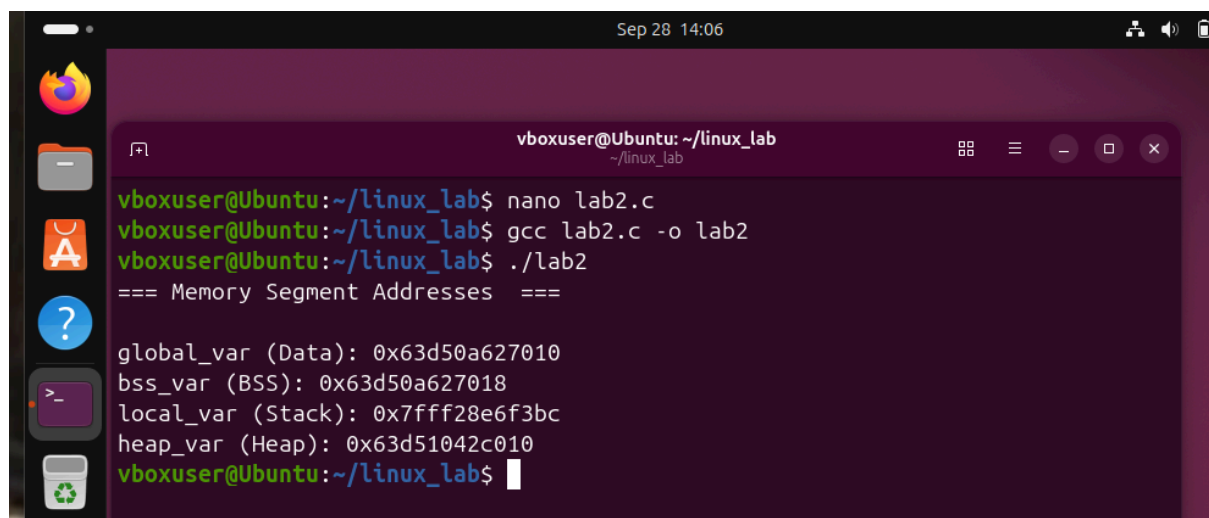
    int *heap_var = malloc(sizeof(int));
    *heap_var = 500;

    printf("=== Memory Segment Addresses ===\n\n");

    printf("global_var (Data): %p\n", (void *)&global_var);
    printf("bss_var (BSS): %p\n", (void *)&bss_var);
    printf("local_var (Stack): %p\n", (void *)&local_var);
    printf("heap_var (Heap): %p\n", (void *)heap_var);
    free(heap_var);
    return 0;
}
```

Figure 4: Code snippet

2) Print the address of each variable using %p



```
vboxuser@Ubuntu: ~/linux_lab
vboxuser@Ubuntu:~/linux_lab$ nano lab2.c
vboxuser@Ubuntu:~/linux_lab$ gcc lab2.c -o lab2
vboxuser@Ubuntu:~/linux_lab$ ./lab2
=== Memory Segment Addresses ===

global_var (Data): 0x63d50a627010
bss_var (BSS): 0x63d50a627018
local_var (Stack): 0x7fff28e6f3bc
heap_var (Heap): 0x63d51042c010
vboxuser@Ubuntu:~/linux_lab$
```

Figure 5: Compilation and running the code

3) Identify which segment has the highest addresses

Answer:

We can identify the segment with the highest addresses by comparing the hexadecimal address values. From the output, the Stack segment has the highest addresses, represented by `local_var` (0x7ffcdaa1304c) and the `heap_var` pointer 0x7ffcdaa13050.

4) Identify which segment has the lowest addresses

Answer:

The Data/BSS segment has the lowest addresses compared with the other memory segments. In the output, `global_var` is located in this lower-address region.

5) Calculate the address difference between Stack and Heap

Answer:

Stack: 0x7ffcdaa1304c

Heap: 0x600eccbe8010

Difference = Stack address – Heap address
= 35,107,295,637,564 bytes

Knowledge Check Questions

1) What is a data type in the context of programming and operating systems?

Answer:

A data type specifies how much memory should be allocated for a variable, how the stored data should be interpreted, and what operations can be performed on that data.

2) Name the five main memory segments in a process memory layout.

Answer:

1. Text – contains machine code/instructions.
2. Data – contains initialized global and static variables.
3. BSS – contains uninitialized global and static variables.
4. Heap – contains dynamically allocated memory.
5. Stack – contains local variables and function calls.

3) What is the difference between the Data segment and the BSS segment?

Answer:

The Data segment stores initialized global and static variables, while the BSS segment stores uninitialized global and static variables.

4) Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

Answer:

The stack is placed at high virtual addresses and grows downward, while the text segment is located at lower addresses. This arrangement provides separate regions for different types of process memory and allows the stack and heap to grow in different directions. The lab identifies the Stack as high-address memory and Data/BSS as low-address memory.

5) What happens to stack memory when a function returns? Who reclaims it?

Answer:

When a function returns, the memory used for its local variables and function call is automatically released from the stack. The compiler/runtime and operating system manage the stack memory, so the programmer does not need to manually free it.

6) How does the OS know which memory belongs to which process?

Answer:

The OS gives each process its own virtual address space and manages the memory assigned to that process. This provides memory isolation, preventing one process from directly accessing another process's memory.

Github Link :

https://github.com/prathanaamgain/PrathanaOS_batch38.git